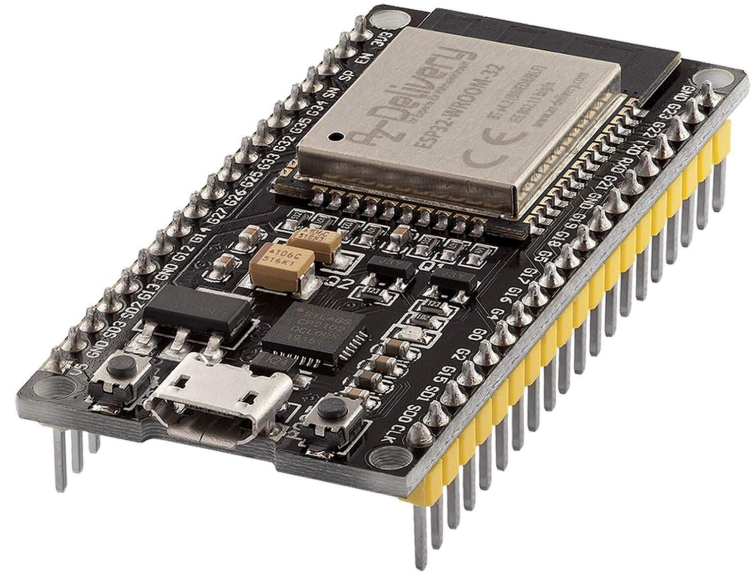




- Introduction to NodeMCU/ESP32, Overview of NodeMCU and its features, Programming NodeMCU via Arduino IDE, Introduction to Wi-Fi Connectivity with Node MCU
- Interfacing LED, Gas Sensor,
-

NodeMCU ESP32 microcontroller

NodeMCU ESP32 is a microcontroller with integrated WiFi and Bluetooth modules. Very easy to use, it's lightweight and has a memory and calculation capacity superior to that of the Arduino. This makes it an ideal board for learning programming, developing connected objects or servers.



NodeMCU Overview



- **What is NodeMCU?**
Open-source Lua-based firmware for **ESP32** and **ESP8266** WiFi SoCs from **Espressif**.
- **Key Features:**
 - Flash-based **SPIFFS** file system.
 - Firmware implemented in **C**.
 - ESP32 version layered over **Espressif ESP-IDF**.
- **History:**
 - Originally a companion for **ESP8266 NodeMCU** modules.
 - Now **community-supported** and runs on **any ESP** module.



NodeMCU

NodeMCU = Node MicroController Unit

An open-source **IoT development platform** based on the **ESP8266** and ESP32 Wi-Fi chip.

- **Built around ESP8266 SoC** from Espressif Systems
- Includes CPU, RAM, Wi-Fi, and SDK
- Ideal for Internet of Things (IoT) projects
- Supports programming in **Lua**, **Arduino C/C++**, and others

Microcontroller features



The NodeMCU ESP32 microcontroller uses the **ESP-WROOM-32 microprocessor (Tensilica Xtensa LX6)**. This processor operates at a clock frequency of 240 MHz, with 520 kB RAM, 448 kB EEPROM and 4000 kB Flash memory (for programming and data logging).

The microcontroller has a WiFi chip, enabling it to connect to the local network, create a server or create its own network for other devices to connect to. The microcontroller has a Bluetooth chip that enables it to interact with other devices.

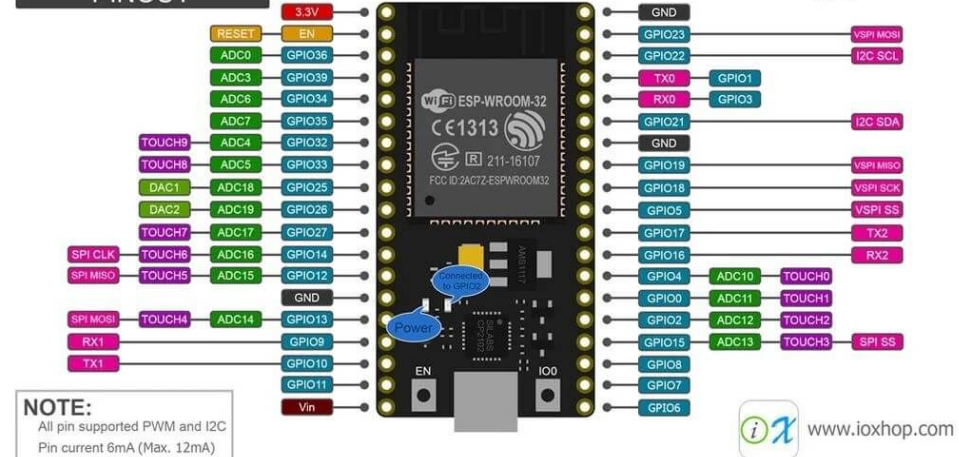
Power supply - The NodeMCU ESP32 microcontroller operates over a voltage range of 7-12V, thanks to its on-board voltage regulator, while the microprocessor operates with a voltage of 3.3V. In normal operation, the microcontroller consumes up to 45mA (if no power is supplied) and can accept a maximum current of 40mA on each of its IO pins.

- CPU ESP-WROOM-32 (Tensilica Xtensa LX6)
- Voltage : 3.3V
- Flash : 4000 kB
- RAM : 520 kB
- EEPROM : 448 kB
- Clock speed : 240MHz
- WiFi : Yes
- Bluetooth : Yes
- SD : No

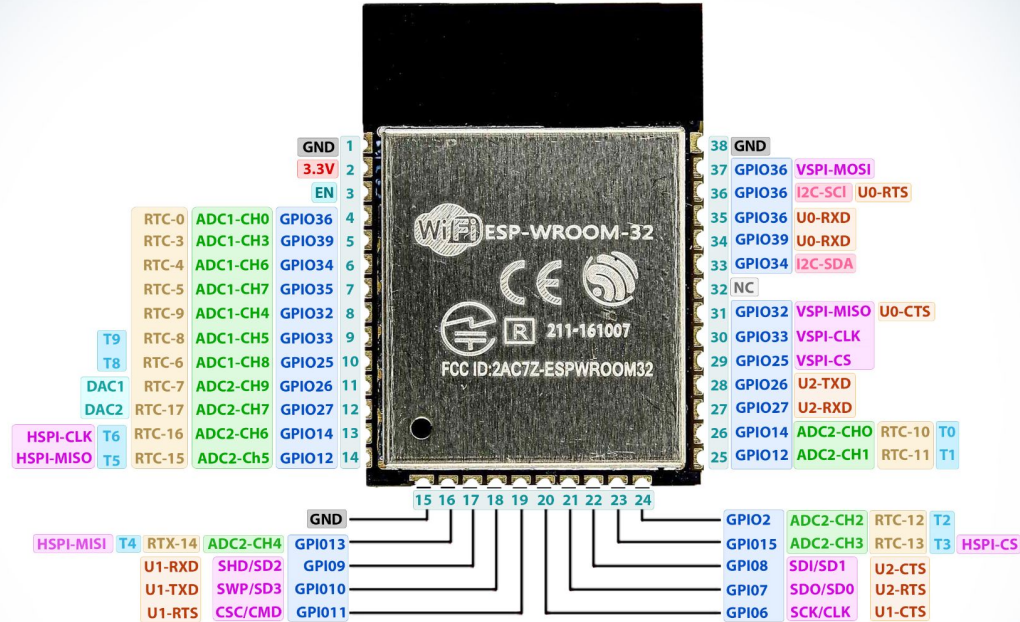
Pin Diagram

- Analog I/O : 15 (2, 4, 12, 13, 14, 15, 25, 26, 27, 32, 33, 34, 35, 36, 39)
- Digital I/O : 4 (all except 6 to 11)
- PWM pins: 16 (2, 4, 5, 6, 12, 13, 14, 15, 18, 19, 21, 25, 26, 27, 32, 33)
- Communication Série : 9 (1, 2, 3, 4, 5, 12, 13, 14, 15, 18, 19, 21, 25, 26, 27, 32, 33)
- I2C communication : 1 (('21', '22'))
- SPI communication: 1 (('5', '18', '19', '23'))
- I2S communication: 1 (('26', '25', '33'))
- Touch: 9 (2, 4, 12, 13, 14, 15, 27, 32, 33)
- Interrupt: 26 (0, 1, 2, 4, 5, 7, 12, 13, 14, 15, 16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33, 34, 35, 36, 3)

NodeMCU-32S
PINOUT



ESP32-WROOM-32 PINOUT



Installing the ESP32 Add-on in Arduino IDE



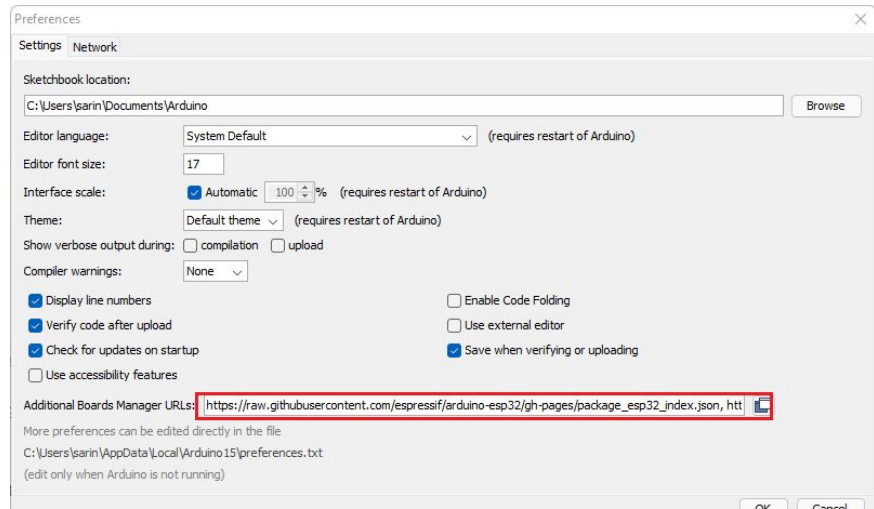
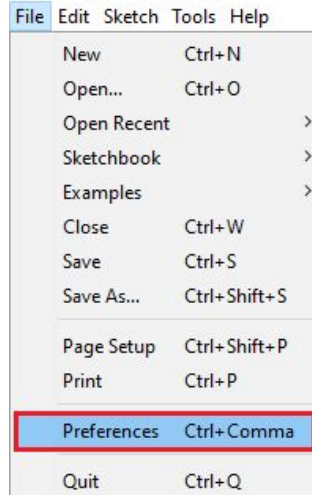
1. Prerequisites & Setup

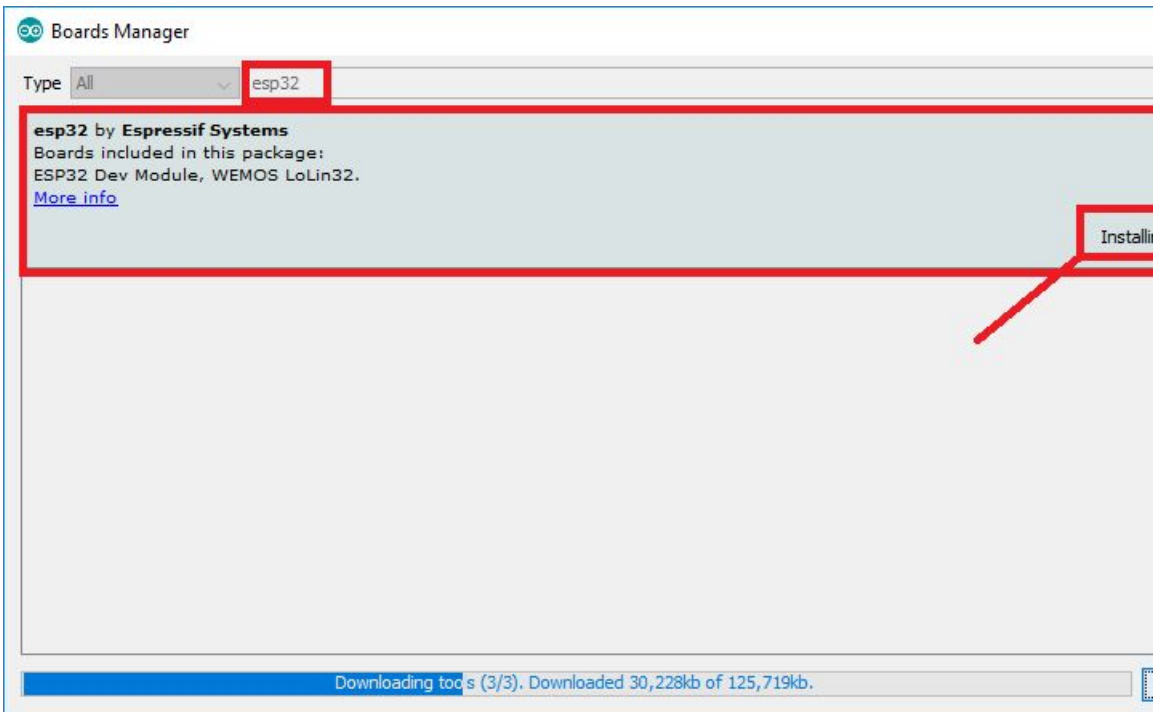
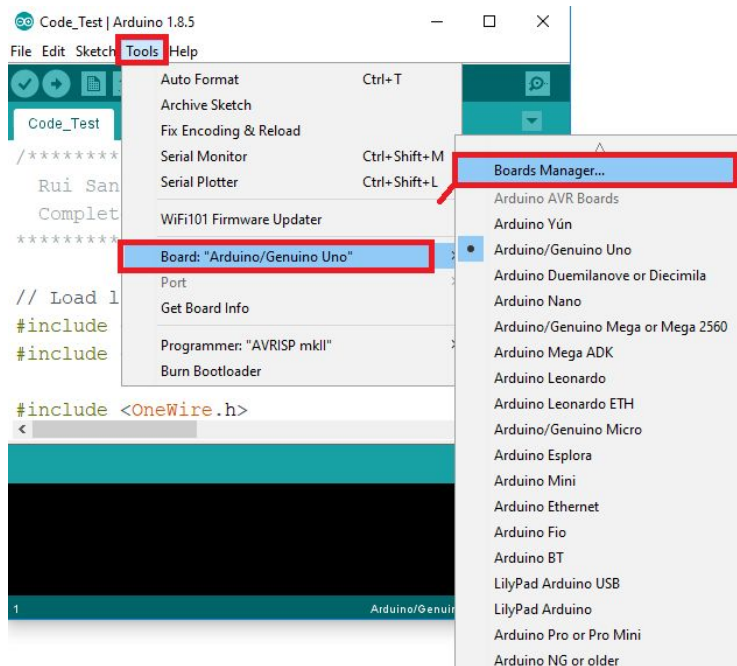
- **Install Arduino IDE** from [arduino.cc](https://www.arduino.cc)
- **Use version 1.8.X** for compatibility with some plugins (e.g., SPIFFS).
- You can also install Arduino 2.0, but some plugins may not work yet.

2. Install ESP32 Add-on

- Open Arduino IDE > File > Preferences
- Paste this URL into “Additional Board Manager URLs”:
 - https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
- Go to Tools > Board > Boards Manager
- Search for ESP32, install “ESP32 by Espressif Systems”

ESP32_data_logging | Arduino 1.8







Boards Manager



Type All ▼

esp32

esp32 by **Espressif Systems** version **1.0.2** **INSTALLED**

Window Snip

Boards included in this package:
ESP32 Dev Module, WEMOS LoLin32.

[More info](#)

Select version ▼

Install

Remove

Close

Testing the Installation and Uploading Code to the ESP32

Now, let's check if the installation was successful and if we can upload new code to the ESP32 board.

We'll simply upload an example sketch from the library of available examples.

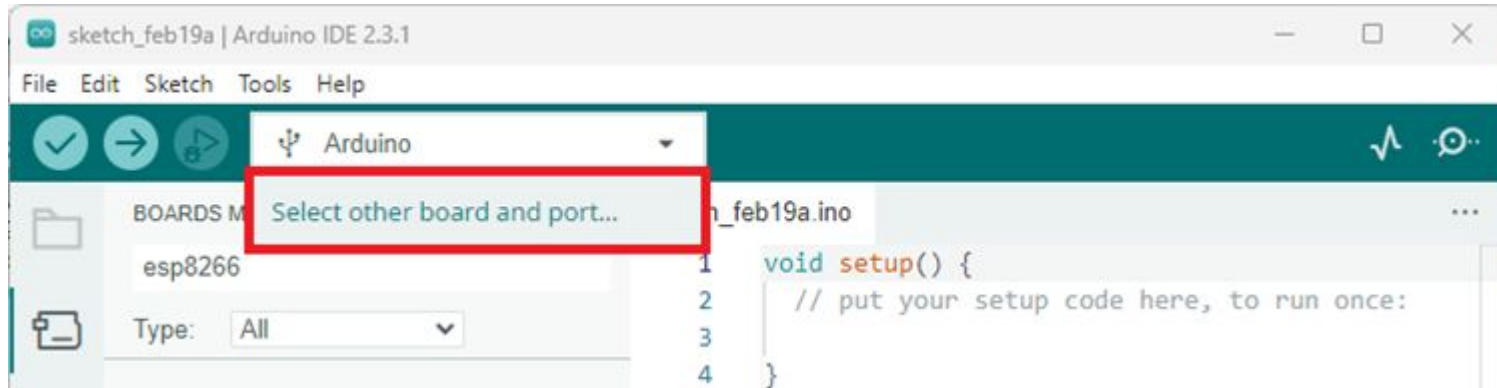
Connect your ESP32 development board to your computer using a USB cable. If you have an ESP32 DEVKIT DOIT board, the built-in red LED will turn on. This indicates the board is receiving power.

Important: you must use a USB cable with data wires. Some USB cables from chargers or power banks are power only and they don't transfer data—these won't work.



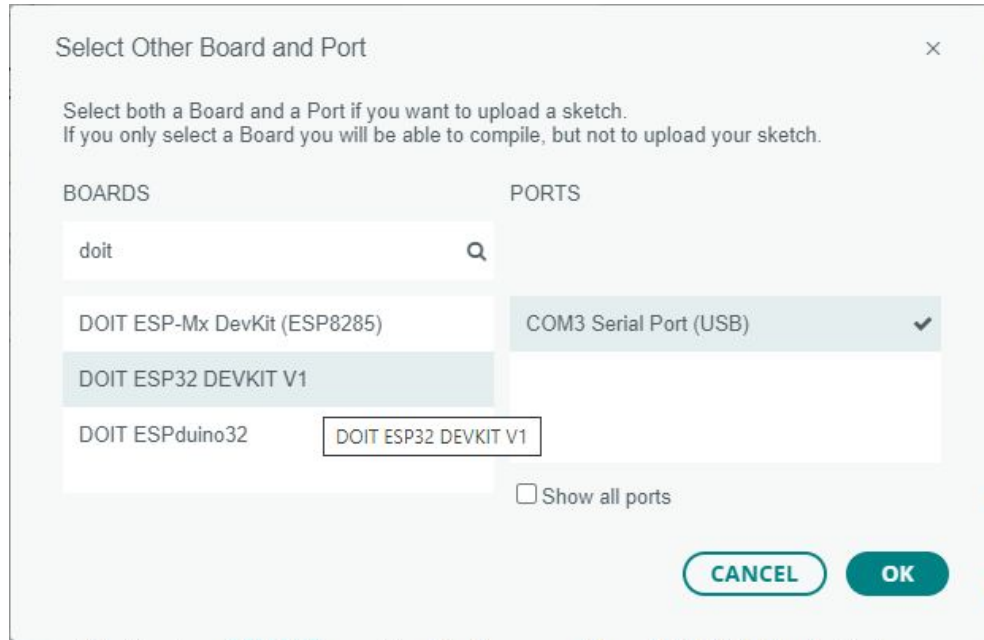
With your Arduino IDE open, follow these steps:

- 1) Select your Board in **Tools** > **Board** menu or on the top drop-down menu, click on “Select other board and port...”



A new window, as shown below, will open. Search for your ESP32 board model.

Select the board model you're using, and the COM port. In our example, we're using the DOIT ESP32 DEVKIT V1. Click **OK** when you're done.





Blink

Blink the blue LED on the ESP-01 module
This example code is in the public domain

The blue LED on the ESP-01 module is connected to GPIO1
(which is also the TXD pin; so we cannot use `Serial.print()` at the same time)

Note that this sketch uses `LED_BUILTIN` to find the pin with the internal LED

*/

```
void setup() {  
  pinMode(LED_BUILTIN, OUTPUT);    // Initialize the LED_BUILTIN pin as an output  
}  
  
// the loop function runs over and over again forever  
void loop() {  
  digitalWrite(LED_BUILTIN, LOW);  // Turn the LED on (Note that LOW is the voltage level  
  // but actually the LED is on; this is because  
  // it is active low on the ESP-01)  
  delay(1000);                     // Wait for a second  
  digitalWrite(LED_BUILTIN, HIGH); // Turn the LED off by making the voltage HIGH  
  delay(2000);                     // Wait for two seconds (to demonstrate the active low LED)  
}
```

Now Click on **Verify** button to compile and check for errors. If compilation is successful without any errors then click on **Upload**.



Step# 4 Testing

Once you upload the code successfully in NodeMCU the blue led will start blinking with a interval of 1 sec. If you get this result then you have successfully uploaded your first code in NodeMCU. Host this was easy tutorial on easy nodemcu programming with simple steps.

Esp32 Connect To Wifi



Esp32 Connect To Wifi

Connect to specific WiFi network and monitor the connection status via serial monitor.

- 
- Start by including the necessary libraries.
 - `#include "WiFi.h"`
 - In the following variables, define your access point network credentials:
 - `const char* ssid = "yourNetworkName";`
 - `const char* password = "yourNetworkPass";`
 - *For example, your wireless SSID is "KakiGodek-WiFi" and your password is "kakigodek123". So you need to replace "yourNetworkName" to "KakiGodek-WiFi" for the SSID, and "yourNetworkPass" to "kakigodek123" for the password.*
 - In the `setup()`, initialize the Serial Monitor for demonstration purposes.
 - `Serial.begin(115200);`
 - Set your ESP32 as an access point with the SSID name and password defined earlier.
 - `WiFi.begin(ssid, password);`

- Then we do a while loop until the connection is establish. We can call the status method on the WiFi object and wait for the result to match the `WL_CONNECTED` enum. We put a small delay to avoid a constant poll.
 - ```
while (WiFi.status() != WL_CONNECTED) {
 delay(500);
 Serial.println("Connecting to WiFi..");
}
```
- When ESP32 successful connected to you WiFi, you can see the result via serial monitor
  - `Serial.println("Connected to the WiFi network");`
  - `// Print local IP address and start web server`
  - `Serial.println("");`
  - `Serial.println("WiFi connected.");`
  - `Serial.println("IP address: ");`
  - `Serial.println(WiFi.localIP());`

# Code :



```
#include "WiFi.h"

const char* ssid = "yourNetworkName"; //choose your wireless ssid
const char* password = "yourNetworkPass"; //put your wireless password here.

void setup() {

 Serial.begin(115200);

 WiFi.begin(ssid, password);

 while (WiFi.status() != WL_CONNECTED) {
 delay(500);
 Serial.println("Connecting to WiFi..");
 }

 Serial.println("Connected to the WiFi network");
 // Print local IP address and start web server
 Serial.println("");
 Serial.println("WiFi connected.");
 Serial.println("IP address: ");
 Serial.println(WiFi.localIP()); //show ip address when connected on serial monitor.
}

void loop() {}
```